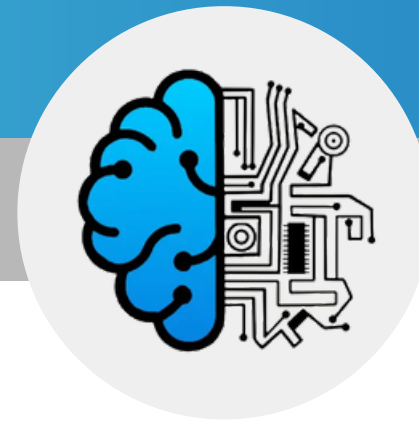




MINICURSO DE ALGORITMOS 2

PET-SIMC

PILHA

PET de Sistemas de Informação do Campus Monte Carmelo

@petsimc



PET de Sistemas de Informação do Campus Monte Carmelo

@petsimc

CRONOGRAMA:

1. Conceitos
2. Sintaxe
3. Exercícios



Introdução

Sempre que escrevemos um programa, é preciso **reservar espaço para as informações que serão processadas.**

Para isso utilizamos as variáveis: Uma variável nada mais é que uma posição de memória que armazena uma informação que pode ser modificada pelo programa.

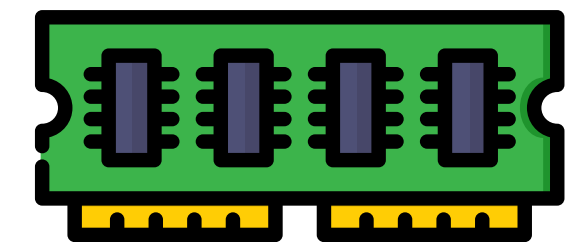


Alocação de memória

É o processo de **reservar espaço na memória do computador** para armazenar dados durante a execução de um programa.

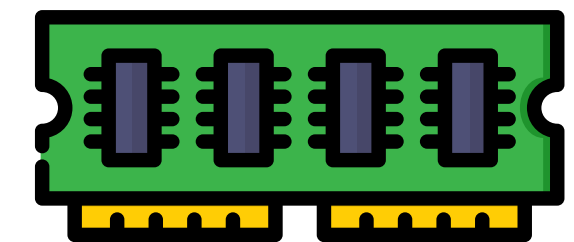
Em C, a memória pode ser alocada de duas formas principais:

- **Estática:** tamanho fixo definido no código.
- **Dinâmica:** espaço reservado durante a execução, usando funções específicas.



Alocação de memória

A alocação dinâmica permite ao programador criar “variáveis” em tempo de execução, ou seja, alocar memória para novas variáveis quando o programa está sendo executado, e não apenas quando se está escrevendo o programa, como a alocação estática.



@petsimc

Compilação x Execução

Compilar: transformar código em linguagem de máquina

É o processo de converter o seu código C (.c) em um executável binário

- Ele verifica se há erros de sintaxe.
- Ele calcula tamanhos de variáveis estáticas, endereços, tipos de dados.
- Ele gera um arquivo binário com instruções que o processador consegue entender.

Compilação x Execução

Executar: rodar o programa compilado

É quando você realmente roda o programa, e o computador segue as instruções que foram geradas na compilação. O sistema operacional carrega o executável na memória

- Ele cria as áreas de memória necessárias (stack, heap, código, etc).
- A função `main()` é chamada, e o código começa a rodar linha por linha.
- Variáveis são criadas, `printf` imprime coisas, `scanf` lê entradas, etc.

Alocação estática

A alocação estática é o que vocês já vinham fazendo desde que começaram a implementar vetores e matrizes...., o espaço é alocado antes do código ser executado.

```
int numeros[5]; // alocação estática: tamanho fixo
```

- O vetor numeros é criado com tamanho fixo
- O tamanho precisa ser conhecido na hora da compilação.
- A memória é liberada automaticamente quando a função termina.

Alocação dinâmica

Na alocação dinâmica, o espaço espaço é reservado durante a execução, usando funções específicas, para economizar memória, alocando só o que for necessário em tempo de execução.

```
int *ptr;  
int n = 5;
```

```
ptr = (int*) malloc(n * sizeof(int)); // aloca espaço para 5 inteiros
```

O malloc(...) aloca um bloco de memória no heap e retorna o endereço do primeiro byte desse bloco, e como você vai guardar um endereço, a variável precisa ser um ponteiro.

Funções de Alocação dinâmica

As funções estão disponíveis na biblioteca `stdlib.h`:

- **`malloc(size_t size)`**: aloca um bloco de memória de tamanho `size` bytes.
- **`realloc(void *ptr, size_t size)`**: realoca um bloco já alocado para um novo tamanho.
- **`free(void *ptr)`**: libera a memória alocada para evitar vazamentos.

```
void *realloc(void *ptr, size_t new_size);
```

```
free(numeros); // Libera memória manualmente
```

O que é uma pilha?



Uma **pilha** (ou stack) armazena uma sequência de valores “empilhados” como em uma pilha de pratos ou de caixas. Uma pilha tem uma base e um topo.

O que é uma pilha?



- A base contém o **primeiro elemento que entrou** na pilha (e que será o **último a sair dela**);
- O topo contém o **último elemento que entrou** na pilha (e **que será o primeiro a sair dela**).

Quais tipos a pilha comporta?

Podemos ter pilhas de **inteiros**, de **racionais**, de **strings**, **structs**, etc.

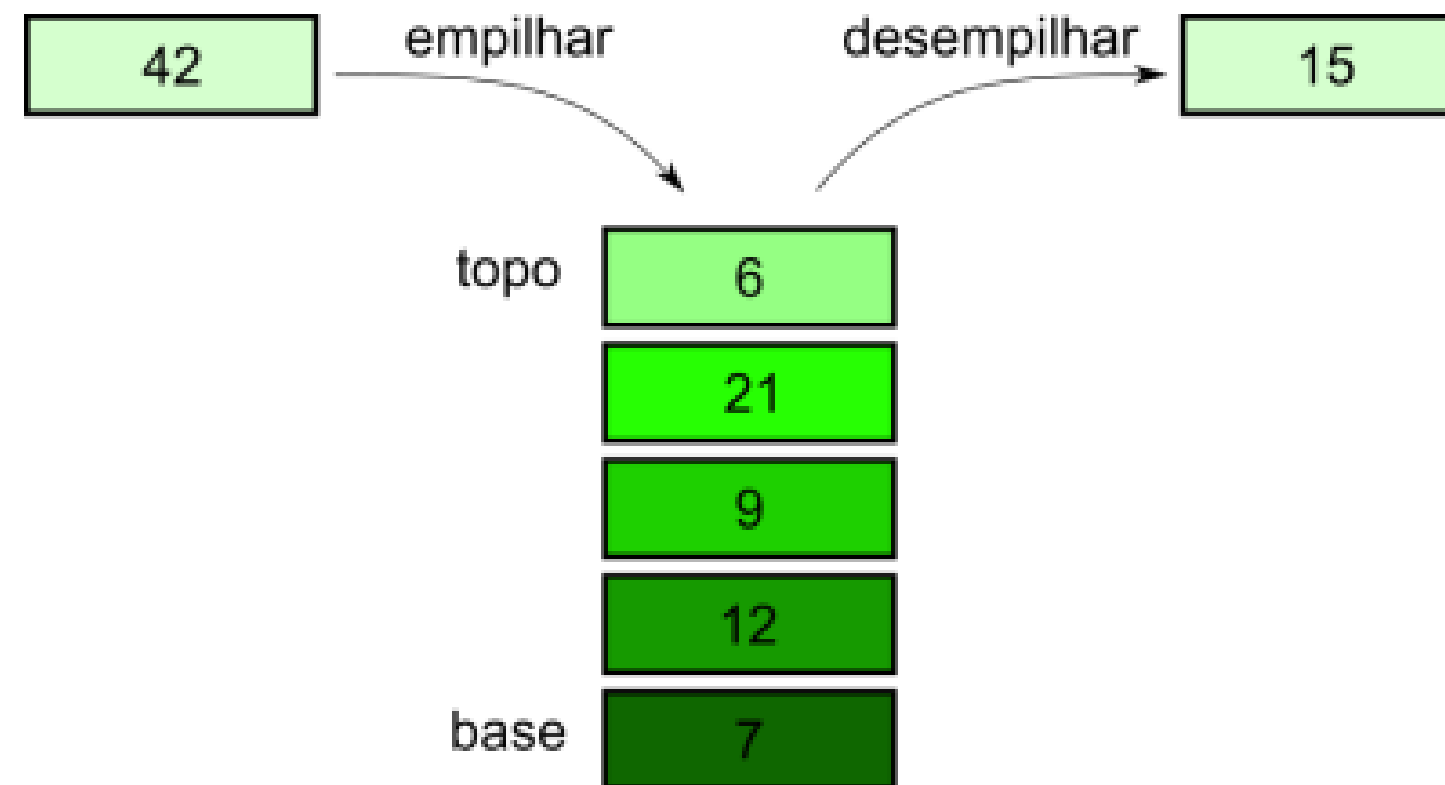


@petsimc

Como funciona?

A principal característica da pilha é que os **elementos entram e saem sempre pelo topo** da pilha.

LIFO, do inglês **Last In, First Out** (o último a entrar é o primeiro a sair).



@petsimc

Como funciona?

Algumas operações definidas sobre uma pilha são:

- `cria (pilha, capacidade)` : cria uma nova pilha, informando o número máximo de elementos que ela pode comportar
- `destroi (pilha)` : esvazia/destrói a pilha
- `insere (pilha, elemento)` : insere um novo elemento **no topo** da pilha
- `elemento = retira (pilha)` : retira o elemento do topo da pilha
- `elemento = topo (pilha)` : devolve o elemento do topo da pilha sem removê-lo
- `tamanho (pilha)` : informa o número atual de elementos na pilha
- ...

Sintaxe:

Uma implementação para o TAD “pilha de números inteiros” em C seria (**pilha.h**):

```
typedef struct pilha Pilha;  
  
Pilha* cria (void);  
void push (Pilha* p, float v);  
float pop (Pilha* p);  
int vazia (Pilha* p);  
void libera (Pilha* p);
```

Sintaxe:

A estrutura que representa o tipo pilha deve, portanto, ser composta pelo vetor e pelo número de elementos armazenados.

```
#define MAX 50

struct pilha {
    int n;
    float vet[MAX];
};
```

@petsimc

Sintaxe:

A função para **criar a pilha** aloca dinamicamente essa estrutura e inicializa a pilha como sendo vazia, isto é, com o número de elementos igual a zero.

```
Pilha* cria (void)
{
    Pilha* p = (Pilha*) malloc(sizeof(Pilha));
    p->n = 0;    /* inicializa com zero elementos */
    return p;
}
```

Sintaxe:

Para **inserir um elemento** na pilha, usamos a próxima posição livre do vetor. Devemos ainda assegurar que exista espaço para a inserção do novo elemento, tendo em vista que trata-se de um vetor com dimensão fixa.

```
void push (Pilha* p, float v)
{
    if (p->n == MAX) {    /* capacidade esgotada */
        printf("Capacidade da pilha estourou.\n");
        exit(1);          /* aborta programa */
    }
    /* insere elemento na próxima posição livre */
    p->vet[p->n] = v;
    p->n++;
}
```

Sintaxe:

A função **pop** **retira o elemento do topo da pilha**, fornecendo seu valor como retorno. Podemos também verificar se a pilha está ou não vazia.

```
float pop (Pilha* p)
{
    float v;
    if (vazia(p)) {
        printf("Pilha vazia.\n");
        exit(1);          /* aborta programa */
    }
    /* retira elemento do topo */
    v = p->vet[p->n-1];
    p->n--;
    return v;
}
```


Sintaxe:

A função que **verifica se a pilha está vazia** pode ser dada por:

```
int vazia (Pilha* p)
{
    return (p->n == 0) ;
}
```

Sintaxe:

Finalmente, a função para **liberar a memória alocada** pela pilha:

```
void libera (Pilha* p)
{
    free(p) ;
}
```

Vamos praticar?!

@petsimc



Exercício 01

Simule as seguintes operações na pilha:

- Insira "Mauricio"
- Insira "Marcelo"
- Remova
- Insira "Guilherme"
- Insira "Paulo"
- Remova
- Insira "João"

Exercício 02

Desenvolva uma operação para transferir elementos de uma pilha P1 para uma pilha P2.

@petsimc

Exercício 03

Crie uma função que retira todos os elementos ímpares da pilha, mantendo a ordem original dos elementos pares da pilha.

@petsimc

Exercício 04

Escreva uma função que receba como parâmetro duas pilhas e verifique se elas são iguais. Duas pilhas são iguais se possuem os mesmos elementos, na mesma ordem.

@petsimc

DESAFIO

Construa um programa utilizando uma pilha que resolva o seguinte problema: Armazene as placas dos carros (apenas os números) que estão estacionados numa rua sem saída estreita. Dado uma placa verifique se o carro está estacionado na rua. Caso esteja, informe a sequência de carros que deverá ser retirada para que o carro em questão consiga sair.

Referências

<http://www2.ic.uff.br/~cbraga/ed/apostila/ed11-pilhas.pdf>

[https://wiki.inf.ufpr.br/maziero/doku.php?
id=progl:tipos_abstratos_de_dados](https://wiki.inf.ufpr.br/maziero/doku.php?id=progl:tipos_abstratos_de_dados)

CONTATO

Como entrar em contato em caso de dúvidas?



E-mail

laviniabd@ufu.br
nicolly.luz@ufu.br



Sala do PET- 1BMC206

UFU - Monte
Carmelo - Campus
Araras



Site do PET-SIMC

petsimc.facom.ufu.br



Grupo do Wpp



@petsimc